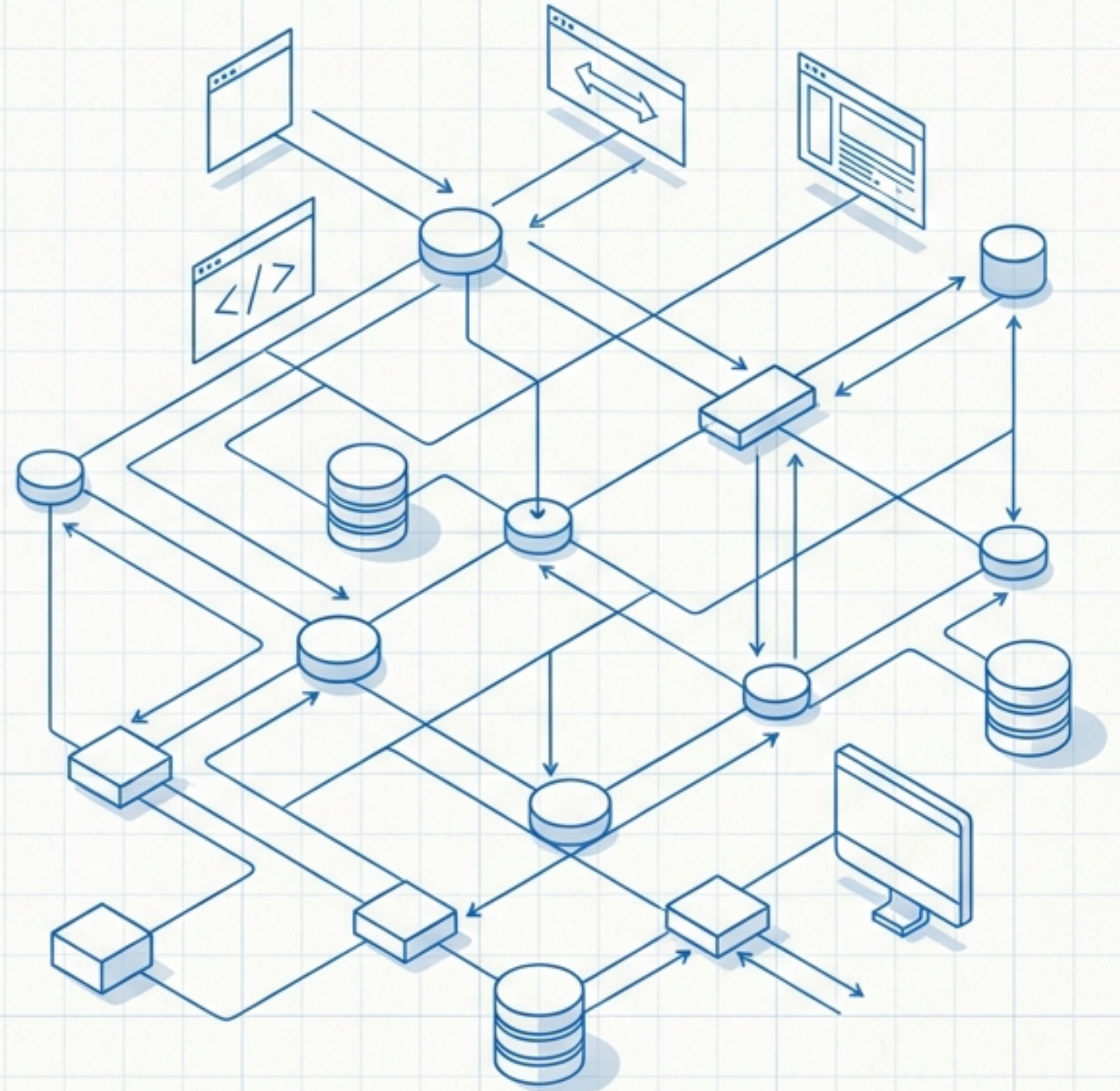


El Atlas de las Arquitecturas de Software

Guía visual de patrones, topologías y flujo de datos para estudiantes de ingeniería.



¿Qué es la Arquitectura de Software?

Es la ciencia que estudia cómo se definen, organizan y comunican los componentes de software, y las restricciones que regulan el sistema.

Clasificación según Bass

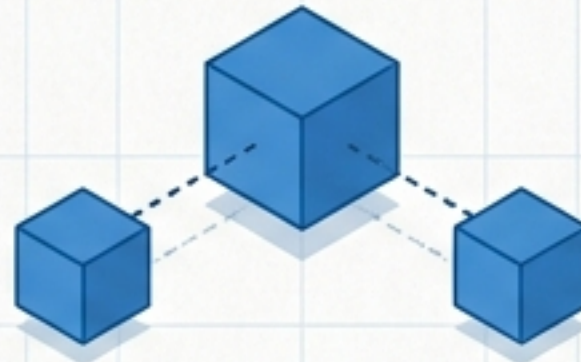


1. Módulos

(Tiempo de diseño)

Muestran la estructura del código.

Ejemplo: Capas



2. Componentes y Conectores [C&C]

(Tiempo de ejecución)

Muestran elementos en acción interactuando.

Ejemplos: Cliente-Servidor, MVC, Broker, P2P



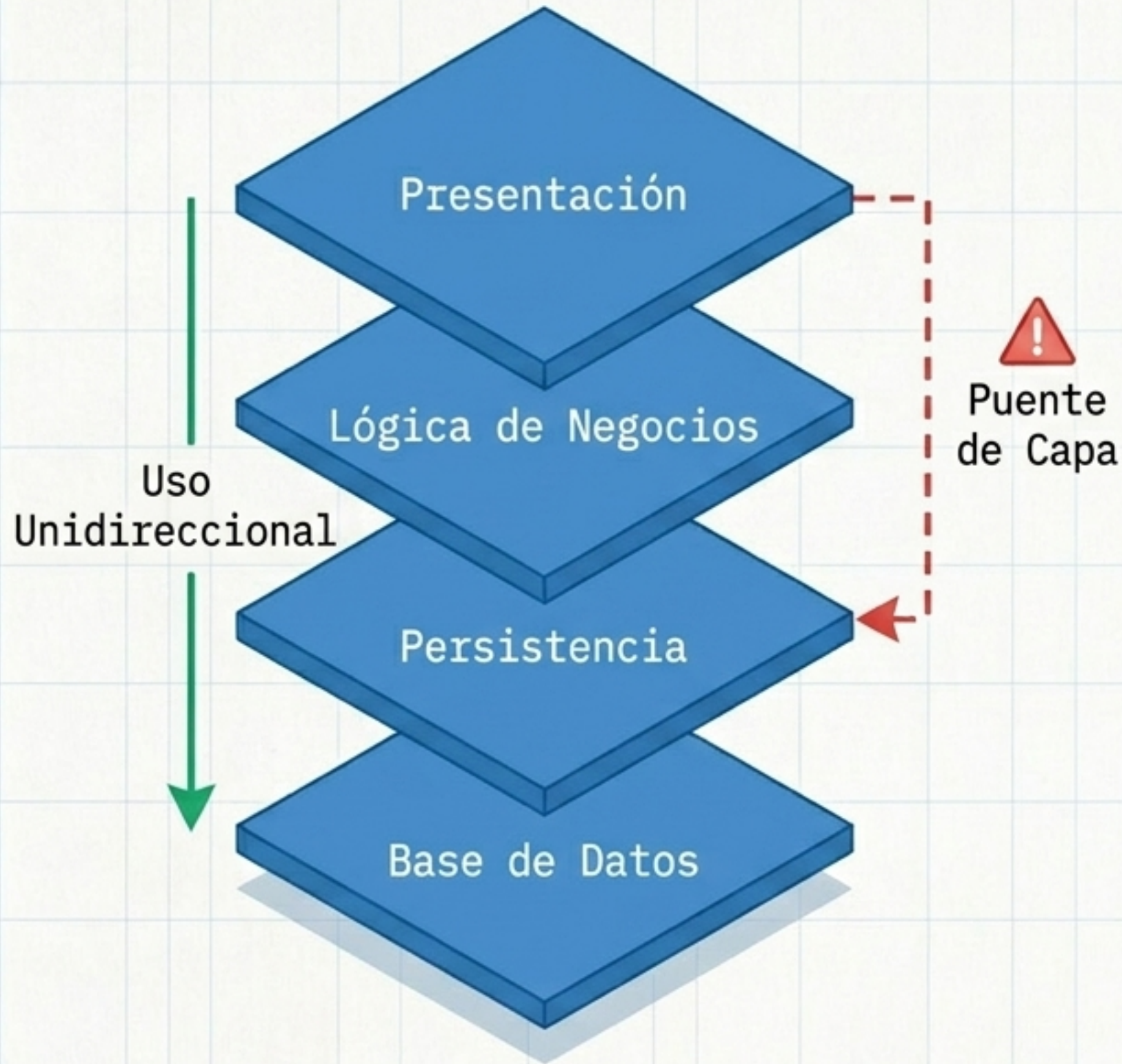
3. Asignación

(Allocation)

Combinan software con hardware (entorno físico).

Ejemplo: Map-Reduce

Arquitectura en Capas (Layers)



Concepto

Divide el software en agrupaciones de módulos cohesivos para separar preocupaciones (Separation of Concerns). Promueve la portabilidad y la modificabilidad.

Key Rule

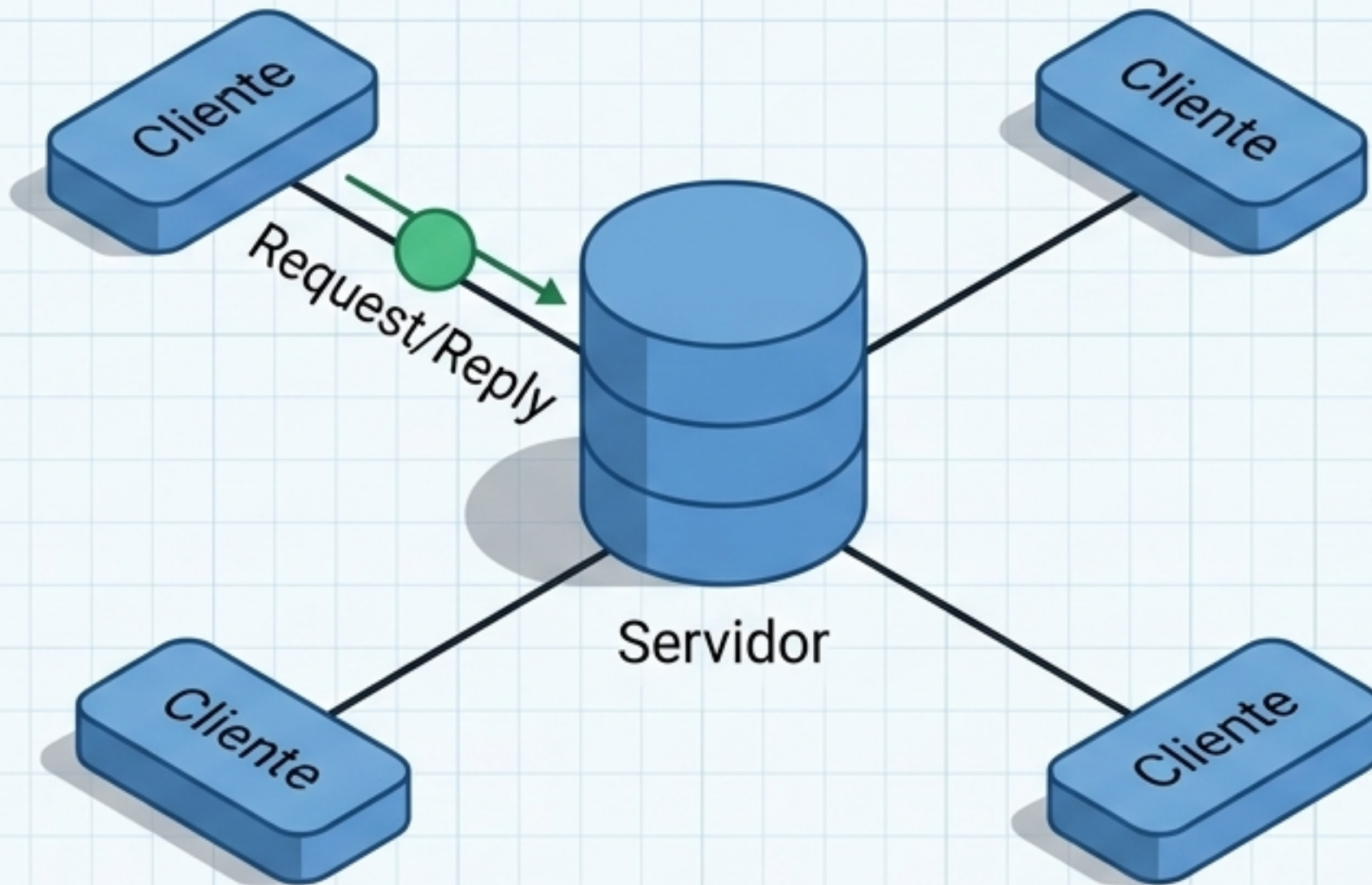
Relación Estricta: Las capas superiores solo pueden usar las instalaciones públicas de la capa inmediatamente inferior. Las relaciones circulares o ascendentes están prohibidas.

Trade-offs & Examples

Trade-offs: Agrega costo inicial y penalización de rendimiento. Demasiados puentes de capa destruyen sus beneficios.

Ejemplos: Modelo OSI, SO THE (Dijkstra).

Cliente - Servidor



1. Concepto:

Un modelo de control centralizado para acceder a recursos compartidos. Los clientes solicitan servicios; un servidor central (o varios distribuidos) los provee.

2. Mecánica:

Utiliza un conector de datos controlado por un protocolo de solicitud/respuesta (ej. HTTP). Puede ser 'sin estado' (stateless).

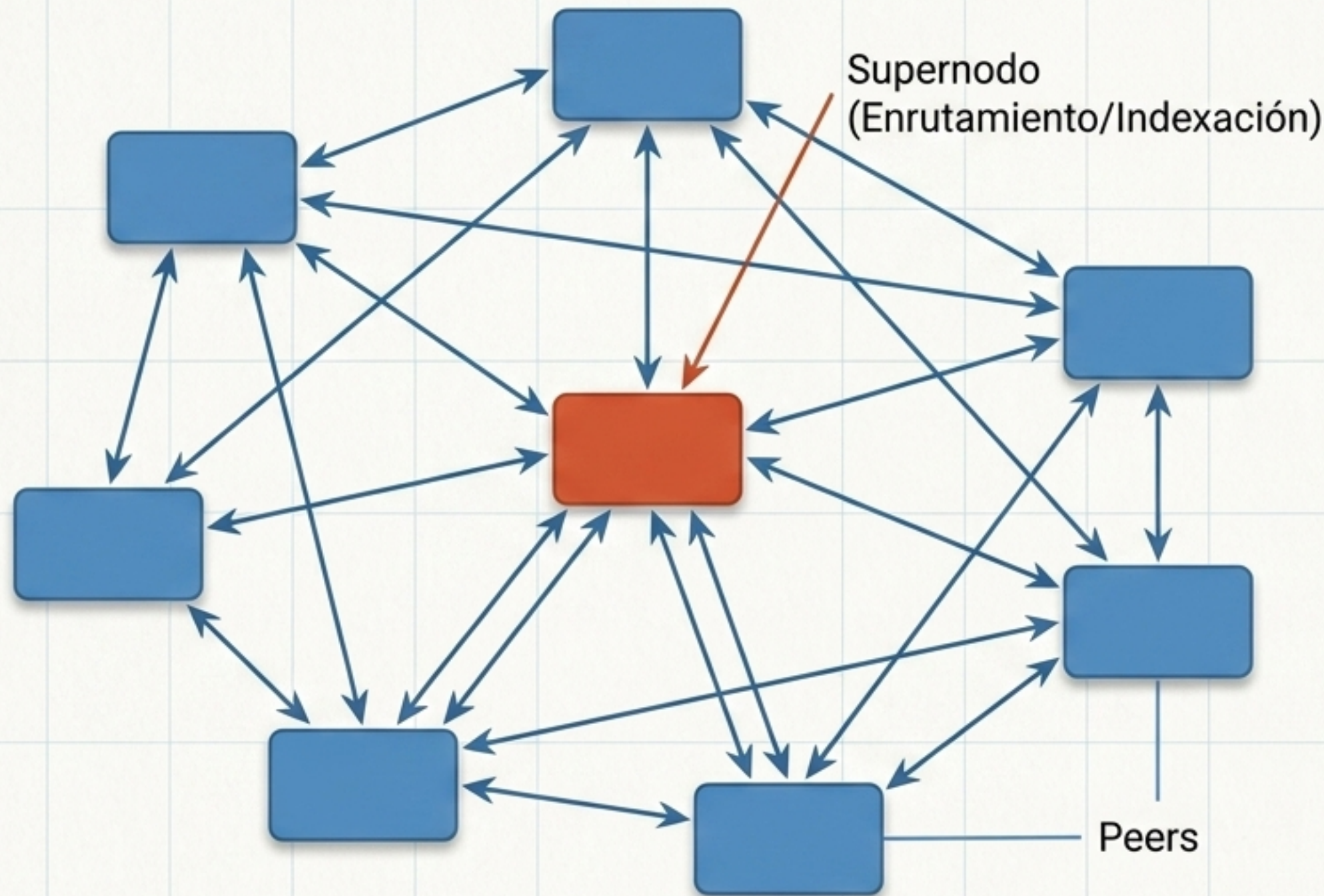
⚠ Trade-offs:

Excelente para centralizar recursos, pero el servidor puede convertirse en un cuello de botella (bottleneck) o un punto único de falla (SPOF).

4. Ejemplo:

La World Wide Web.

De Igual a Igual (Peer-to-Peer / P2P)



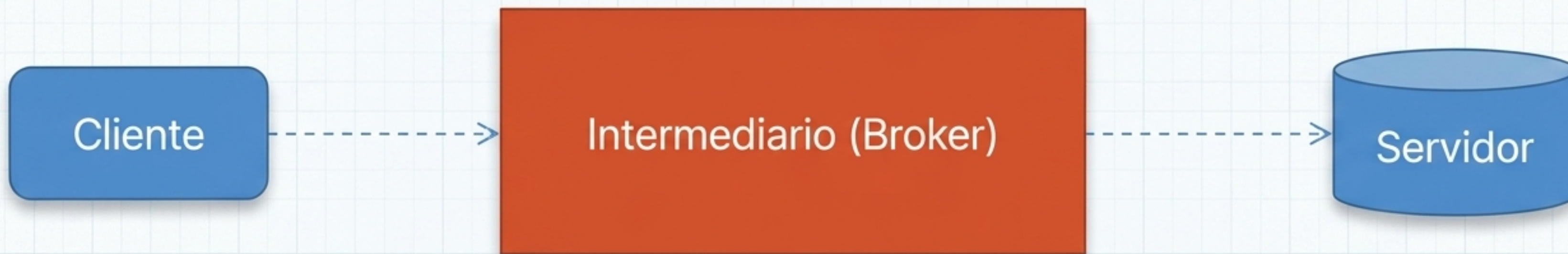
Concepto: Una red descentralizada donde las entidades computacionales interactúan directamente como "iguales".

Mecánica: No hay asimetría. Cada nodo (peer) actúa simultáneamente como cliente y servidor. Si un nodo cae, el sistema sobrevive (alta disponibilidad).

Trade-offs: Escala masivamente, pero la seguridad, coherencia y recuperación de datos son sumamente complejas comparadas con Cliente-Servidor.

Ejemplos: BitTorrent, Skype, Gnutella.

Intermediario (Broker)



Concepto:

Desacopla clientes y servidores. El cliente nunca conoce la ubicación o identidad del servidor real.

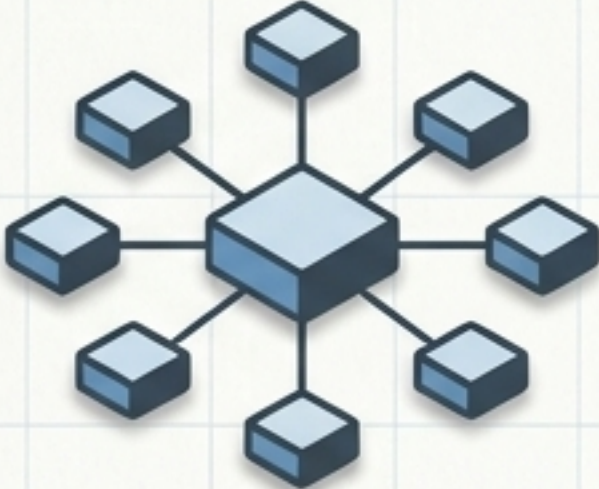
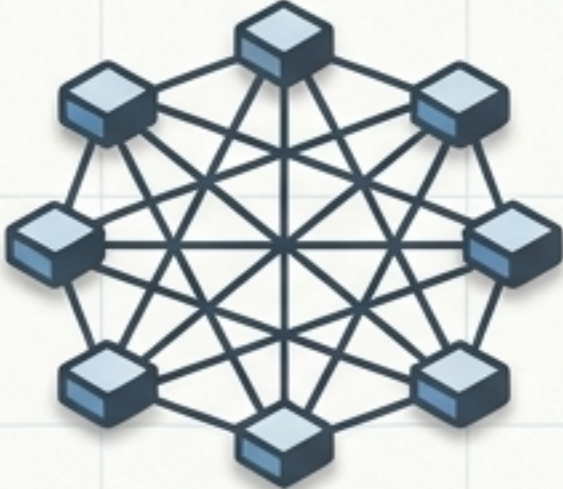
Mecánica:

El cliente solicita un servicio al Broker. El Broker localiza un servidor adecuado, reenvía la solicitud, recibe la respuesta y se la devuelve al cliente.

Trade-offs:

Facilita cambiar servidores dinámicamente. Agrega latencia y el Broker mismo puede ser un punto de falla único.
Ejemplo: Sistemas Proxy.

⚔ Comparativa: Topologías de Red

Cliente-Servidor	P2P	Broker
		
Control: Centralizado. Fallo: El servidor detiene todo (SPOF). Uso Ideal: Recursos compartidos controlados (ej. Web HTTP).	Control: Descentralizado. Fallo: Altamente resistente (no hay SPOF). Uso Ideal: Colaboración de recursos masivos (ej. BitTorrent).	Control: Desacoplado/Intermediado. Fallo: El Broker es el eslabón débil. Uso Ideal: Sistemas distribuidos dinámicos donde los servidores cambian.

Usa C-S para control, P2P para resiliencia masiva, y Broker para flexibilidad y anonimato entre partes.

Tuberías y Filtros (Pipes & Filters)



Concepto: Transforma flujos de datos discretos mediante pasos sucesivos e independientes.

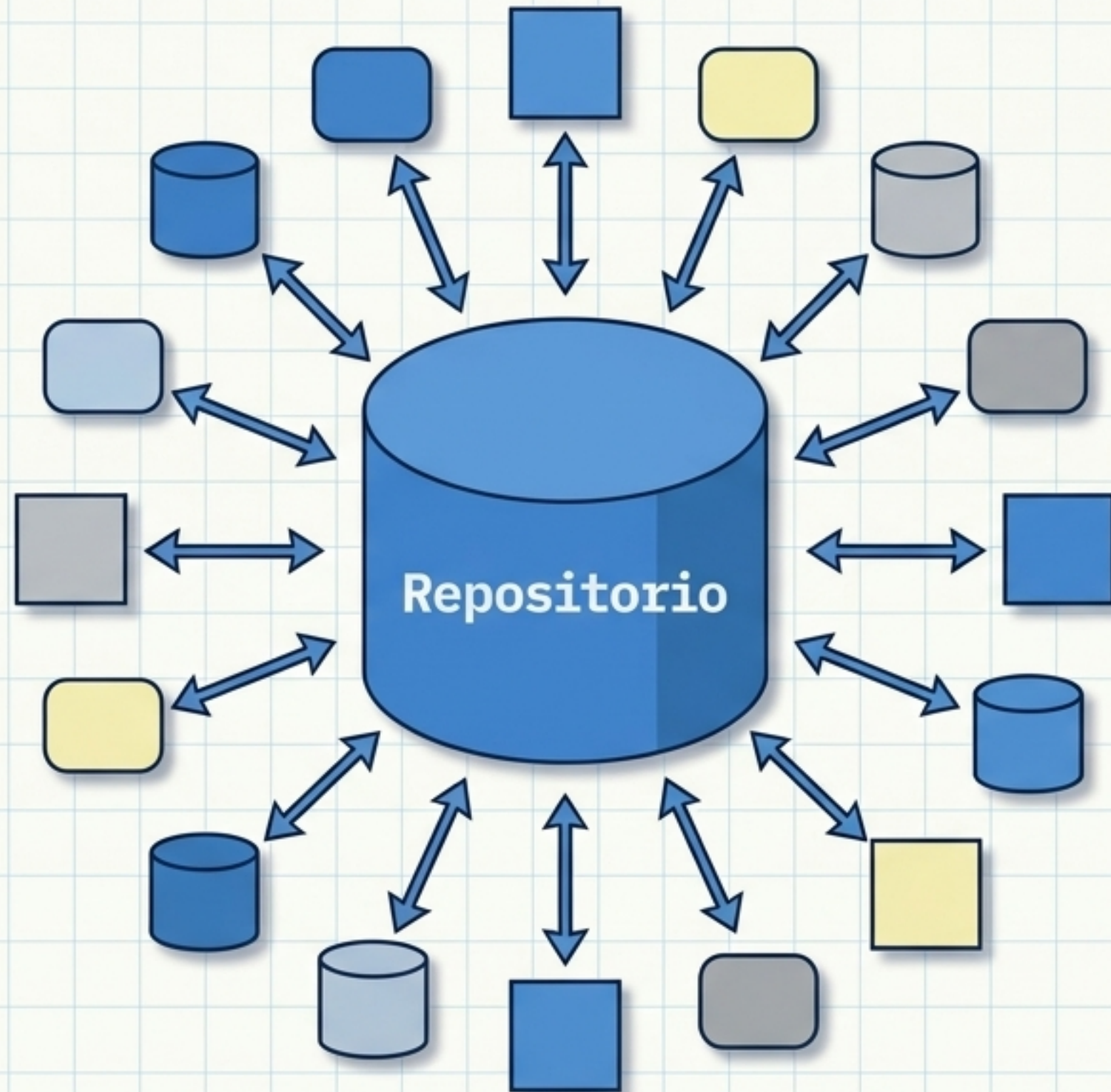
Filtros: Transforman los datos. Ejecutan en paralelo o secuencial.

Tuberías: Conectores que transportan datos sin alterarlos.

Trade-offs: Excelente para procesar datos modulares y reutilizables. Mala elección para sistemas interactivos (no hay ciclos de retroalimentación para el usuario).

Ejemplos: Procesamiento Apache, Yahoo RSS feeds.

Repositorio (Datos Compartidos)



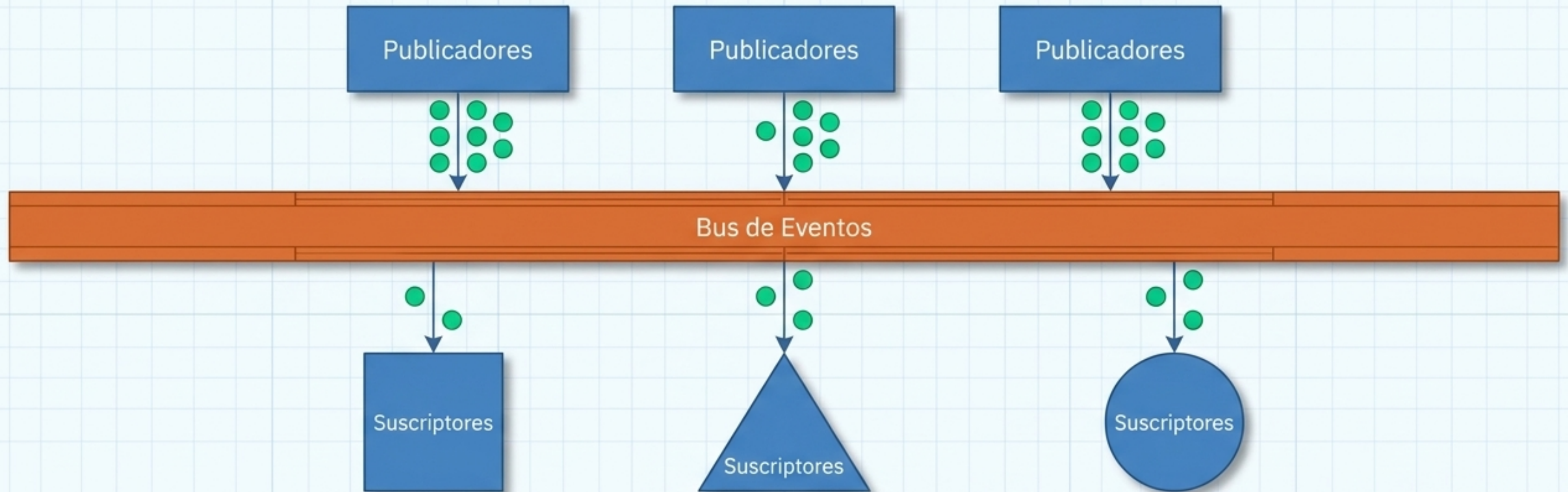
Concepto: Múltiples componentes independientes necesitan acceder y manipular un gran estado de datos compartidos.

Mecánica: La interacción está dominada por el intercambio de datos persistentes. Los componentes no interactúan entre sí, sino únicamente a través del almacén central.

Trade-offs: Los productores y consumidores están fuertemente acoplados al modelo de datos. El repositorio es el **único punto de falla** y un **gran cuello de botella**.

Ejemplos: Sistemas de control de versiones como SVN, GitHub, GitLab.

Publicador - Suscriptor (Pub-Sub)



Concepto: Integración asíncrona donde productores y consumidores de datos interactúan sin conocer la existencia del otro.

Mecánica: Los Publicadores anuncian eventos al Bus. El Bus distribuye estos eventos a los Suscriptores ("fire-and-forget").

Ejemplo: Apache Kafka.

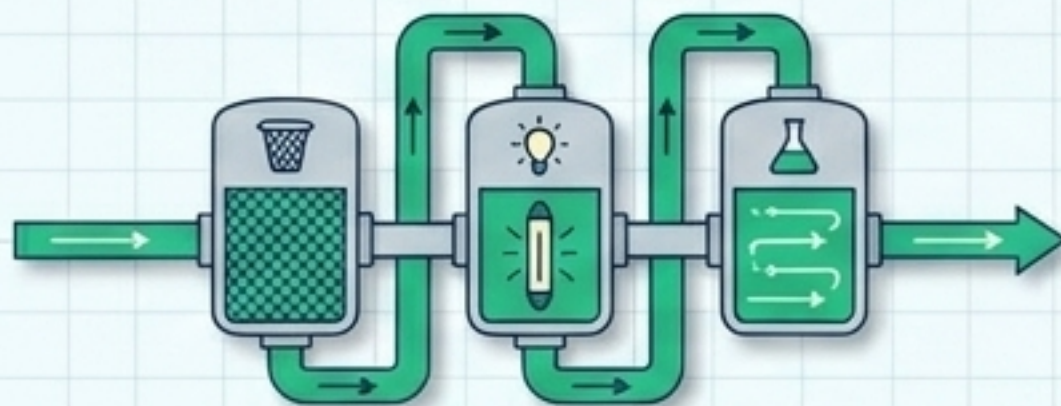
Trade-offs: Escalabilidad masiva y acoplamiento nulo.

Contras: No hay garantía de entrega, sin control de orden, introduce latencia incierta. No apto para tiempo real estricto.



Comparativa: El Flujo de los Datos

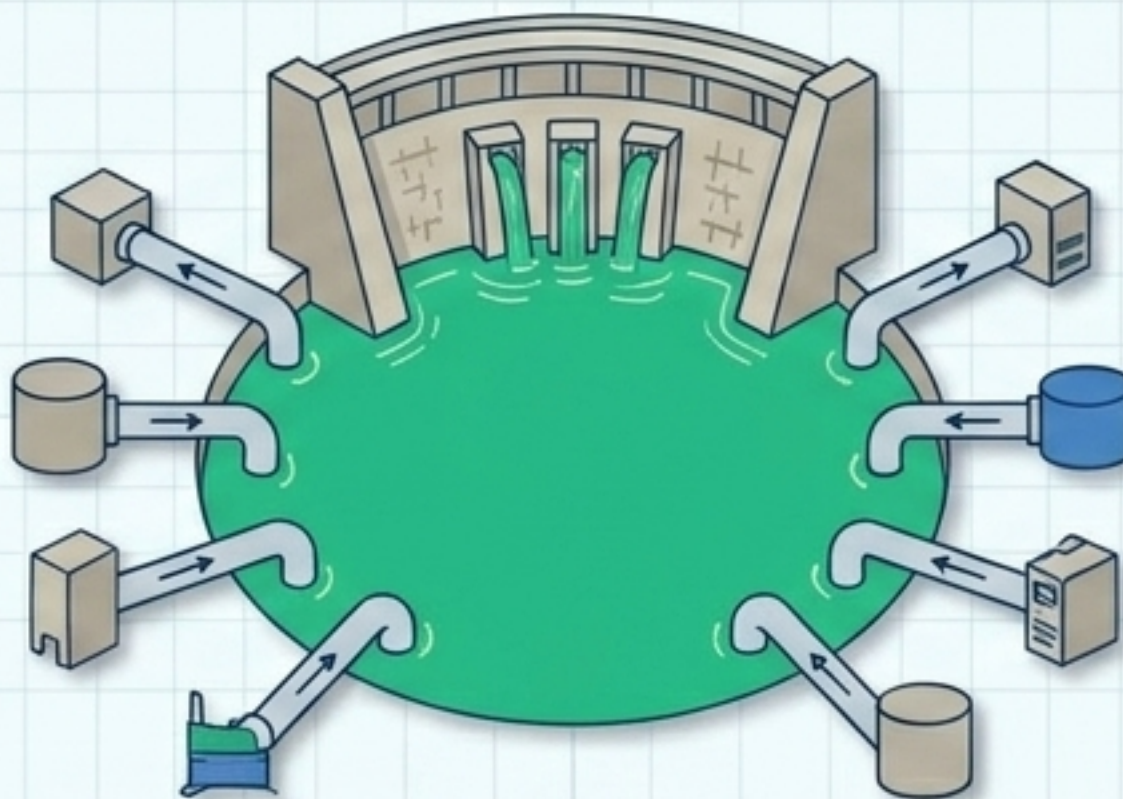
Tuberías y Filtros



Metáfora: La Planta Purificadora.

Enfoque: Procesamiento en tránsito continuo. El agua fluye y se transforma secuencialmente.

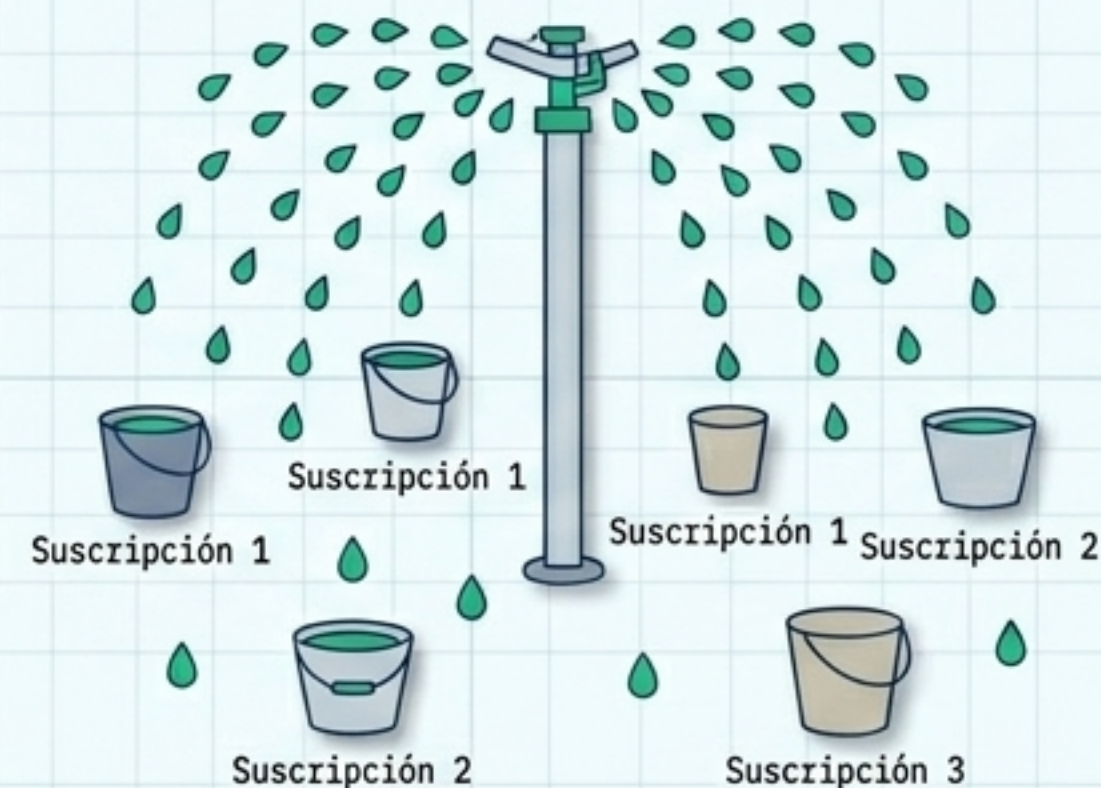
Repositorio



Metáfora: La Represa.

Enfoque: Estado persistente, compartido y centralizado. El agua se almacena en un punto donde todos acuden a beber.

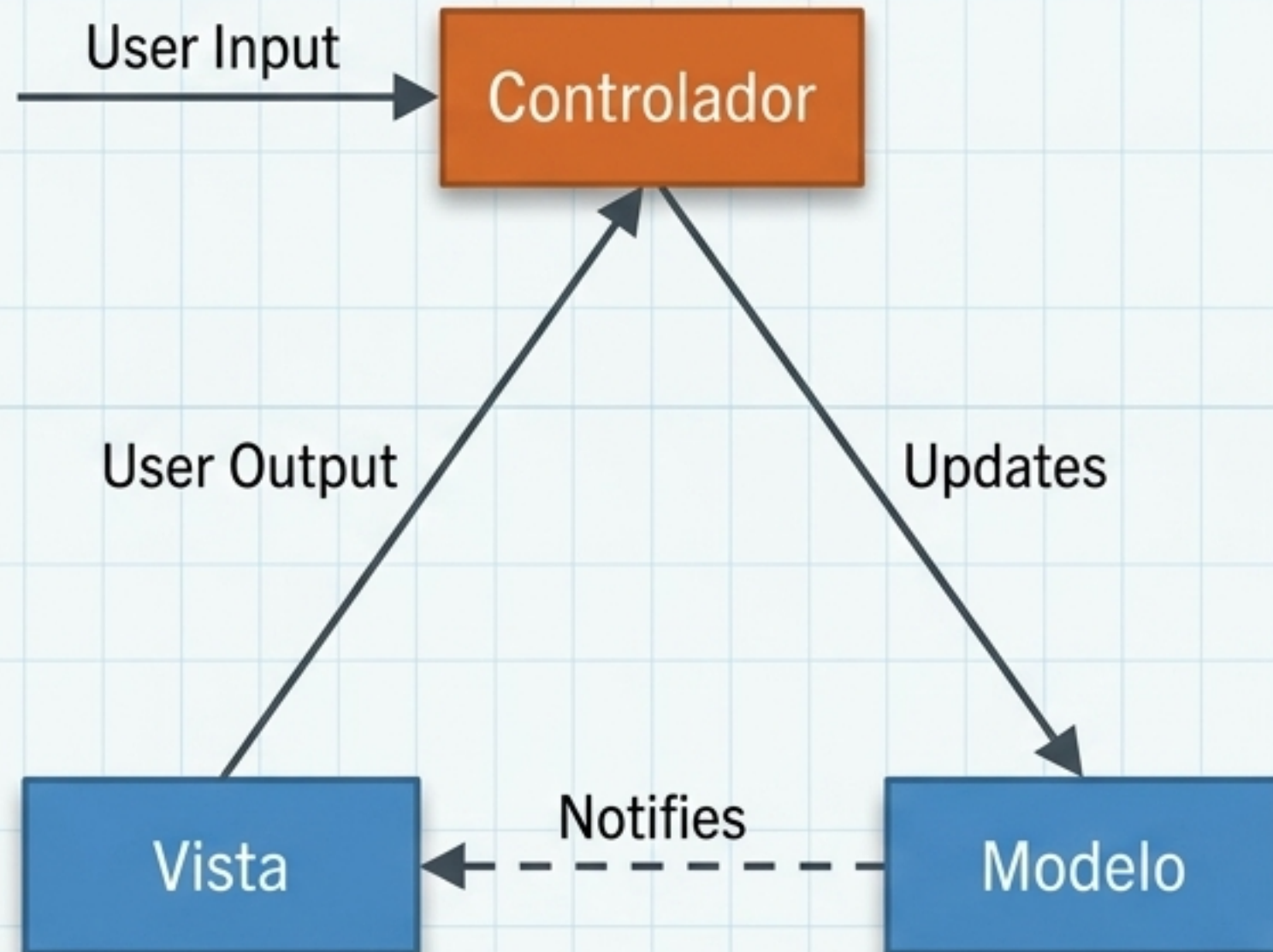
Pub-Sub



Metáfora: Sistema de Riego.

Enfoque: Notificación asíncrona. El agua se dispara al aire y quien haya puesto un balde (suscripción) la atrapa.

Modelo - Vista - Controlador (MVC)



Concepto: Separa la interfaz de usuario de la lógica de la aplicación para aislar modificaciones.

Modelo: Estado y lógica de los datos.

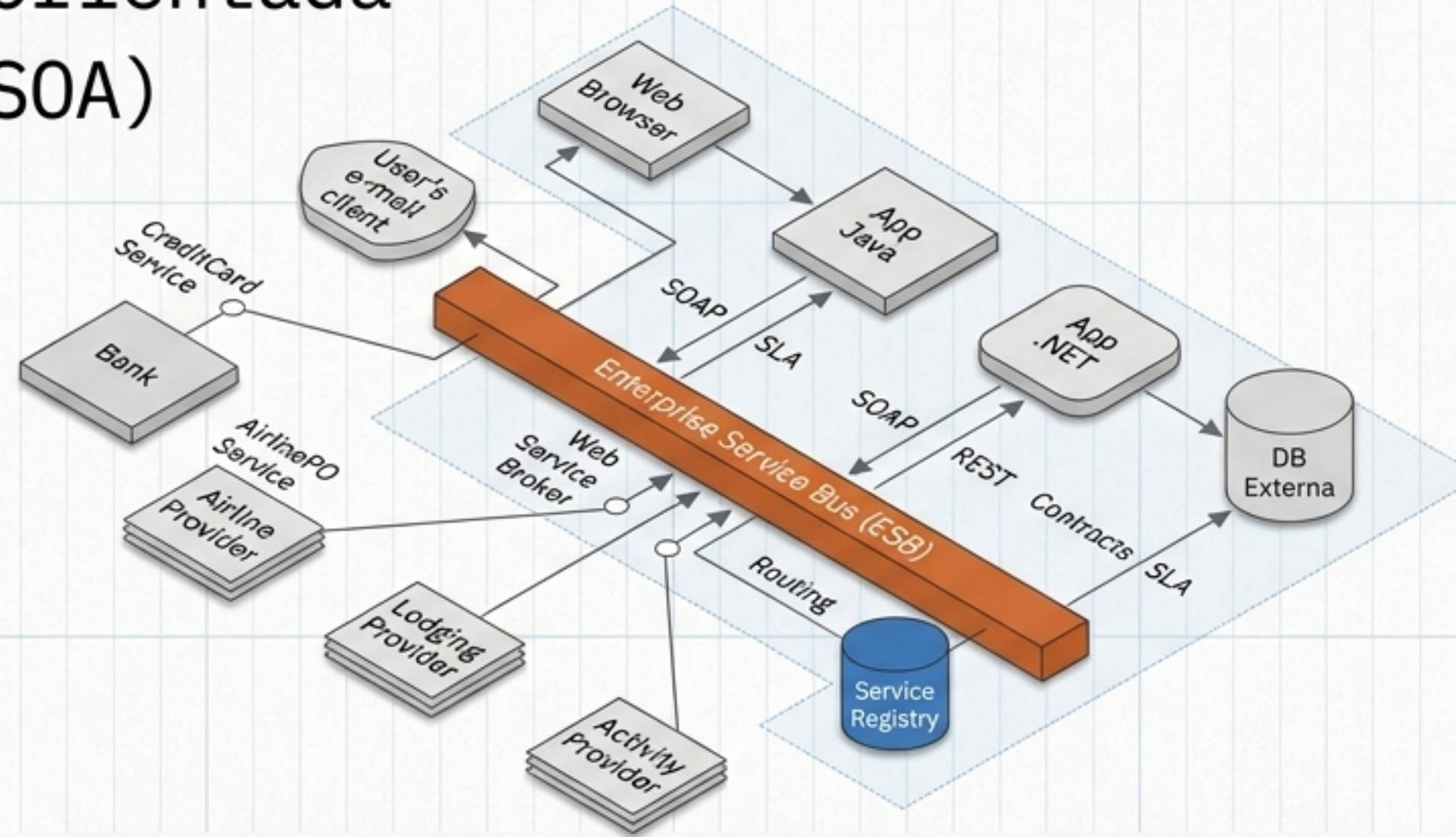
Vista: Representación visual e interfaz interactiva.

Controlador: Traduce la acción del usuario en cambios del modelo/vista.

Trade-offs: Demasiada complejidad y costo para interfaces simples. Discrepancia conceptual con algunos toolkits visuales.

Ejemplos: Java Swing, ASP.NET, Qt.

Arquitectura Orientada a Servicios (SOA)



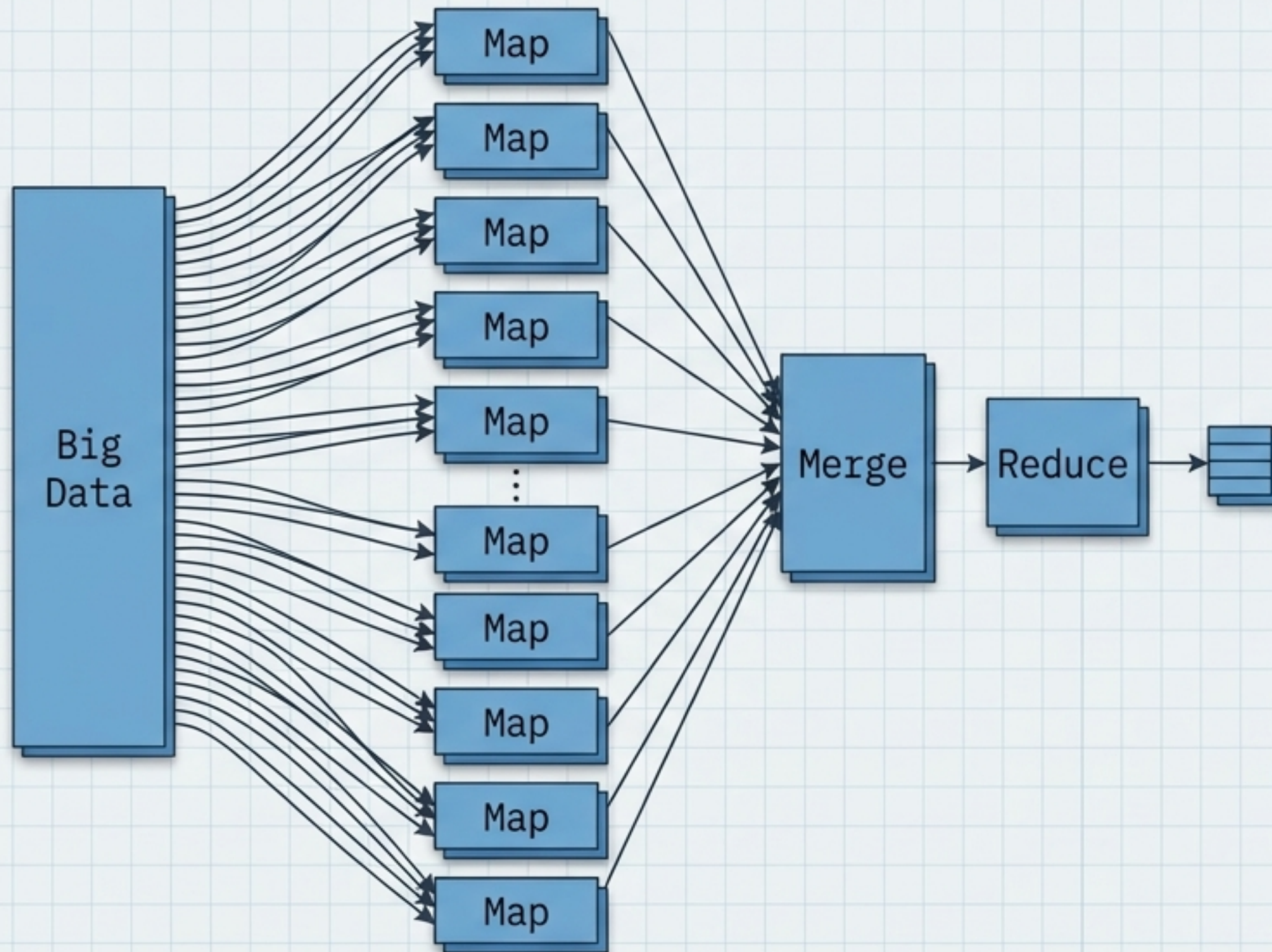
Concepto: Colección de componentes distribuidos (generalmente de diferentes organizaciones o lenguajes) interoperando por la web.

Mecánica: Proveedores publican servicios mediante contratos (SLA) y registros. Consumidores invocan usando protocolos estándar (SOAP, REST). Usa un ESB para enrutar mensajes.

Trade-offs: Permite interoperabilidad masiva multiplataforma. Son sistemas complejos, con sobrecarga de middleware y dependencia de la evolución de servicios ajenos.

Ejemplos: Amazon Web Services, AFIP WS.

Asignación: Map-Reduce



Concepto: Arquitectura para clasificar y analizar volúmenes gigantescos de datos (Petabytes) de manera distribuida y paralela.

Map (Mapear): Extrae y transforma datos en paralelo a través de múltiples procesadores.

Reduce (Reducir): Consolida y carga los resultados filtrados.

Trade-offs: Alta resiliencia a fallos de hardware (si un nodo falla, el trabajo se reasigna). Diseñado puramente para Big Data.

Ejemplo: Apache Hadoop.

Matriz de Decisión del Arquitecto

Si tu problema es...	Solución
Separar el código para mantenimiento independiente.	→ Capas
Proteger recursos centrales consultados por muchos.	→ Cliente-Servidor
Alta disponibilidad descentralizada.	→ Peer-to-Peer
Procesar datos en múltiples pasos sucesivos.	→ Tuberías y Filtros
Eventos asíncronos para sistemas no acoplados.	→ Pub-Sub
Desacoplar la UI de la lógica de negocio.	→ MVC
Interoperabilidad empresarial de servicios.	→ SOA
Analizar Petabytes de datos en paralelo.	→ Map-Reduce

El diseño arquitectónico no es sobre encontrar el patrón perfecto, sino sobre elegir el compromiso (trade-off) adecuado.